

## Monte Carlo Pi - Threading Assignment - Answers

### Part 1: The Phantom Bug

```
PS C:\Users\ACYC\Downloads\InClassPractice3week> java PhantomBug
Run 1: pi ~= 1,205881 (hits=15073512, time=376 ms)
Run 2: pi ~= 1,172233 (hits=14652911, time=324 ms)
Run 3: pi ~= 1,273654 (hits=15920675, time=414 ms)
Run 4: pi ~= 1,268737 (hits=15859212, time=376 ms)
Run 5: pi ~= 1,226743 (hits=15334290, time=366 ms)
PS C:\Users\ACYC\Downloads\InClassPractice3week>
```

Run 1: pi ~= 1.212101 (hits=15151259, time=394 ms)  
Run 2: pi ~= 1.201175 (hits=15014688, time=393 ms)  
Run 3: pi ~= 1.285677 (hits=16070966, time=353 ms)  
Run 4: pi ~= 1.332347 (hits=16654335, time=384 ms)  
Run 5: pi ~= 1.265725 (hits=15821564, time=350 ms)

Observation: pi should be about 3.14, but every run comes out between 1.2 and 1.35, and the value is different each time. The 4 threads are all doing `totalHits++` on the same shared variable with no protection. `totalHits++` is really three separate steps (read, add 1, write back), so when two threads interleave those steps, one thread's increment gets overwritten and silently lost. With 4 threads racing on 50,000,000 points, most of the increments get lost, which is why the count (and therefore pi) comes out roughly 2.5x too low, and why it's not even the same wrong answer twice.

### Part 2: The Synchronization Trap

```
PS C:\Users\ACYC\Downloads\InClassPractice3week> java SyncTrap
Single-threaded:      pi ~= 3,141609, time = 574 ms
4-thread (AtomicLong): pi ~= 3,141433, time = 881 ms
Multi-threaded is 1,53x the single-threaded time (i.e. it got SLOWER)
```

Single-threaded: pi ~= 3.141749, time = 436 ms  
4-thread (AtomicLong): pi ~= 3.141518, time = 816 ms  
Multi-threaded is 1.87x the single-threaded time (it got SLOWER)

Observation: switching to `AtomicLong` fixes the correctness problem, pi is now accurate. But the 4-thread version took almost twice as long as one plain core. Every `incrementAndGet()` call forces all 4 cores to fight over exclusive ownership of the same tiny piece of memory, so the cores spend nearly all their time on coordination overhead instead of actual math.

### Part 3: OpenMP-Style Reduction

Multi-threaded is 1.95x the single threaded time (i.e. it got SLOWER)

```
PS C:\Users\ACYC\Downloads\InClassPractice3week> java OpenMPReduction
```

| Threads | Runtime(ms) | Speedup | Efficiency | pi estimate |
|---------|-------------|---------|------------|-------------|
| 1       | 1202        | 1,00x   | 100,0%     | 3,141620    |
| 2       | 591         | 2,03x   | 101,7%     | 3,141904    |
| 4       | 323         | 3,72x   | 93,0%      | 3,141708    |
| 8       | 249         | 4,83x   | 60,3%      | 3,141748    |
| 16      | 228         | 5,27x   | 32,9%      | 3,141593    |
| 32      | 224         | 5,37x   | 16,8%      | 3,141386    |

| Threads | Runtime(ms) | Speedup | Efficiency |
|---------|-------------|---------|------------|
| 1       | 1602        | 1.00x   | 100.0%     |
| 2       | 1257        | 1.27x   | 63.7%      |
| 4       | 630         | 2.54x   | 63.6%      |
| 8       | 444         | 3.61x   | 45.1%      |
| 16      | 395         | 4.06x   | 25.3%      |
| 32      | 436         | 3.67x   | 11.5%      |

Question 1: Why didn't 16 threads run twice as fast as 8?

Going from 8 to 16 threads barely improved runtime (444 ms -> 395 ms, only about a 12% gain) instead of doubling. This points to the CPU having somewhere around 4 physical cores. Threads 1-4 scale almost perfectly (2.54x speedup, ~64% efficiency), but from 4 to 8 the gains shrink (efficiency drops to 45%), and from 8 to 16 they nearly vanish (efficiency drops to 25%). That pattern matches hyperthreading: beyond the number of physical cores, extra "threads" are just sharing the same physical core's execution units and cache, so they add a little throughput but nowhere near double. At 32 threads, runtime actually gets worse again (436 ms) because now there are far more threads than the hardware can truly run at once, so the OS spends extra time context-switching between them instead of doing useful work.

Question 2: Why was the synchronized/AtomicLong version in Part 2 slower than one single core?

With 4 threads all calling incrementAndGet() on the exact same shared counter at extremely high frequency, every single increment needs exclusive ownership of that memory location before it can proceed. Since all 4 threads want it constantly, ownership has to keep bouncing between cores' caches (cache-line ping-pong), and each bounce costs far more cycles than the increment itself would on its own. So instead of splitting the work 4 ways, most of the "work" being done is actually just cores waiting on and passing around a lock, which is why it ended up 1.87x slower than doing the whole thing on a single thread with no

locking at all.